Domain-specific area and objectives of the project

This project investigates laptop pricing in the consumer electronics market using a real world dataset of laptop specifications and prices. Laptops are important devices for education, professional work, and personal use, and their prices vary alot depending on hardware parts like RAM capacity, processor type, storage configuration, and display features. However, these specifications can sometimes presented inconsistantly, making it difficult to understand which features really influences price differences. Gaining some insight into these relationships can be valuable for consumers making decisions and for sellers and manufacturers to price products competitively.

The dataset used in this project was initially selected due to its relevance to this domain and the availability of detailed specification and pricing information. Its suitability for linear regression was not assumed. Instead, exploratory data analysis was carried out before model development to assess whether a linear regression approach was appropriate. This involved visualising laptop price against key numerical features, such as RAM size and storage capacity, and examining correlation values between features and price. These analyses revealed broadly increasing trends and moderate linear associations for several variables, indicating that laptop price exhibits sufficiently linear behaviour to justify the use of linear regression as a baseline modelling approach.

Linear regression is particularly suitable for this task because laptop price is a continuous numerical variable and many of the explanatory features can be represented numerically after preprocessing. In addition, linear regression offers a high degree of interpretability: the effect of each feature on price can be examined directly through model coefficients, rather than relying on complex hard to understand models whose predictions are difficult to explain. This interpretability is especially important in a consumer electronics context, where understanding why a device is priced higher is often as important as the prediction itself.

The primary objective of this project is to analyse how different laptop specifications influence price and to develop a regression model capable of predicting laptop prices based on these features. A further objective is to identify and rank the most influential features contributing to price variation. Through data cleaning, feature extraction from unstructured text columns, statistical analysis, and visualisation, this project demonstrates how linear regression and regularised variants can be applied to noisy, real world consumer data. The findings highlight both the strengths and limitations of linear models in this domain and provide insights that can support data driven decision making in product pricing.

Dataset description

The dataset used in this project is the Uncleaned Laptop Price Dataset from Kaggle [1]. It contains pricing and specification information for more than 1300 laptop models, making it a medium sized dataset suitable for both exploratory analysis and regression

modelling. The dataset is provided as a single CSV file and represents real world electronics data collected from online retail listings. However it reflects inconsistencies and noise that are usually found in commercial product data.

The dataset consists of numerical and categorical features. The target variable, Price, is a continuous numerical value most probably in Indian Rupees. Other numerical attributes include RAM capacity (in GB), screen size (in inches), and laptop weight. Some important predictors are initially are hidden inside the text, such as Memory and CPU, where multiple attributes are combined into a single string. For example, memory specifications may appear as "512GB SSD + 1TB HDD", and processor details may include brand, model, and speed in the same column. Categorical variables describe specifications such as laptop brand, operating system, processor family, graphics information, and display. The dataset also contains missing or irregular values, including non numeric symbols in the weight column, which require cleaning prior to analysis.

This dataset is suitable for the coursework because it does not initially satisfy First Normal Form, with multiple pieces of information stored within individual fields. This enables the demonstration of data preprocessing techniques, including handling missing values, extracting numerical values from text, splitting combined columns into separate features, and converting data types for statistical analysis.[3]

[2] The dataset is also appropriate for linear regression analysis. The target variable is continuous, and most explanatory features can be represented numerically after preprocessing, such as RAM size, storage capacity, processor speed, screen size, and weight. Subsequent exploratory analysis and visualisation in this project show that several of these features exhibit structured and approximately linear relationships with price. While not all relationships are perfectly linear, the data displays sufficient linear behaviour to justify the use of linear regression as an interpretable baseline model. The moderate dataset size further supports meaningful statistical analysis and visualisation without excessive computational cost.

Despite its suitability, the dataset has limitations that should be acknowledged. Prices reflect online retail listings and may differ from final transaction prices or prices in other geographic markets. Additionally, extracting features from unstructured text fields introduces potential noise and information loss. The dataset also lacks temporal information, such as the year of release, which could influence pricing trends. These limitations do not prevent effective regression analysis but help contextualise model performance and motivate the use of regularisation and careful feature engineering later in the project.

Data preparation

Data Loading & Inspection

In [7]:
```
## Import pandas dataframe to open and read the csv
import pandas as pd
## Import re for pattern matching (extract or clear values from a text )  regex
import re
```

```
## Import numpy for numerical operations and handle missing values
import numpy as np
```

I used pandas to load and manipulate the CSV dataset, NumPy for handling missing values and numeric operations, and re known as regular expressions to later extract values from text based fields such as CPU and Memory.

In [9]:
```
# # read the csv and show the first 5 rows of the dataframe
df = pd.read_csv("laptopData.csv")
df.head()
```

Out[9]:

	Unnamed: 0	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory
0	0.0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8GB	128GB SSD
1	1.0	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8GB	128GB Flash Storage
2	2.0	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD
3	3.0	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD
4	4.0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8GB	256GB SSD

The dataset was loaded from the CSV file into Pandas Dataframe

In [11]:
```
# analyse the shape and the data information ( data type )
df.info()
df.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Unnamed: 0        1273 non-null   float64
 1   Company           1273 non-null   object 
 2   TypeName          1273 non-null   object 
 3   Inches            1273 non-null   object 
 4   ScreenResolution  1273 non-null   object 
 5   Cpu               1273 non-null   object 
 6   Ram               1273 non-null   object 
 7   Memory            1273 non-null   object 
 8   Gpu               1273 non-null   object 
 9   OpSys             1273 non-null   object 
 10  Weight            1273 non-null   object 
 11  Price             1273 non-null   float64
dtypes: float64(2), object(10)
memory usage: 122.3+ KB
```

Out[11]: (1303, 12)

I inspected the dataset to look through the data dimensions , data types and see which columns require data type conversions. Most columns are stored as object as even the columns like RAM and Wight which are supposed to be numeric have the text "GB" , "kg" hence cleaning is needed before statistical analysis , visualisation and modelling

In [13]:
```
# check for missing values
df.isna().sum()
```

Out[13]:
```
Unnamed: 0          30
Company             30
TypeName            30
Inches              30
ScreenResolution    30
Cpu                 30
Ram                 30
Memory              30
Gpu                 30
OpSys               30
Weight              30
Price               30
dtype: int64
```

I check for missing values for each column. Each column has 30 missing values which suggests that missing data occurs probably in the same rows instead of being scattered across the different columns

In [15]:
```
# show the the first 5 rows and the missing data rows
df[df.isna().any(axis=1)].head()
```

Out[15]:

	Unnamed: 0	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
46	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
115	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
125	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

I displayed a sample of the rows that have missing values and these rows are fully blank for all columns indicating missing data entry

In [17]:
```
# print the rows before and after dropping
print("Before dropping empty rows:", df.shape)
df = df.dropna()
print("After dropping empty rows:", df.shape)
```

```
Before dropping empty rows: (1303, 12)
After dropping empty rows: (1273, 12)
```

The rows dont have usable features or even price labels hence dropping them is appropriate and it avoids imputing values that may not be known for the entires. I recorded the data size before and after to display the change

In [19]:
```
# remove the index column
df = df.drop(columns=["Unnamed: 0"])
df.head()
```

Out[19]:

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8GB	128GB SSD	Intel Iris Plus Graphics 640	r
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8GB	128GB Flash Storage	Intel HD Graphics 6000	r
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD	Intel HD Graphics 620	I
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD	AMD Radeon Pro 455	r
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8GB	256GB SSD	Intel Iris Plus Graphics 650	r

The Unamed: 0 column is an index column and it does not have anything to do with the laptop specifications and could mislead the model and hence its removed

In [21]:
```
df.info()
df.isna().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1273 entries, 0 to 1302
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Company           1273 non-null   object 
 1   TypeName          1273 non-null   object 
 2   Inches            1273 non-null   object 
 3   ScreenResolution  1273 non-null   object 
 4   Cpu               1273 non-null   object 
 5   Ram               1273 non-null   object 
 6   Memory            1273 non-null   object 
 7   Gpu               1273 non-null   object 
 8   OpSys             1273 non-null   object 
 9   Weight            1273 non-null   object 
 10  Price             1273 non-null   float64
dtypes: float64(1), object(10)
memory usage: 119.3+ KB
```

Out[21]:
```
Company             0
TypeName            0
Inches              0
ScreenResolution    0
Cpu                 0
Ram                 0
Memory              0
Gpu                 0
OpSys               0
Weight              0
Price               0
dtype: int64
```

After basic cleaning and inspection the dataset only contains valid and useful rows ready for detailed cleaning and feature extraction

Cleaning

In [24]:
```
# change 2 in 1 Convertible to just " Convertible "
df["TypeName"] = df["TypeName"].str.replace("2 in 1 Convertible", "Convertible",
```

I standardised TypeName to reduce duplicate category labels if any (eg. 2 in 1 Convertible to Convertible), which prevents unnecessary separations when encoding categorical variables.

In [26]:
```
# round of the inches data to 1 dp as there was inconsistant figures .
# errors='coerce' is there to fill in NAN if  value cannot be converted . This h
df["Inches"] = pd.to_numeric(df["Inches"], errors="coerce").round(1)
# drop missing values
df = df.dropna(subset=["Inches"])
df = df[(df['Inches'] >= 10) & (df['Inches'] <= 20)]
```

Inches was converted to numeric so it can be used as a regression feature. One invalid entry became NaN and was removed

In [28]:
```
# locate the column ram and remove GB from it , and missing values or values tha
# case is false because we want to ignore the upper or lower case in gb when rem
#it can see it as plain text and not a regular expression
df["Ram_GB"] = pd.to_numeric(df["Ram"].astype(str).str.replace("GB", "", regex=F
df = df.drop(columns=["Ram"])
df = df.dropna(subset=["Ram_GB"])
df["Ram_GB"] = df["Ram_GB"].astype("Int64")
```

RAM values were stored as text (e.g., '8GB'). I extracted the numeric value into Ram_GB to make it usable for statistical analysis and regression

In [30]:
```
# remove the word kg from weight and round off the values to 2 decimal place for
df["Weight_kg"] = pd.to_numeric(df["Weight"].astype(str).str.replace("kg", "", r
df = df.drop(columns=["Weight"])
df = df.dropna(subset=["Weight_kg"])
```

I converted weight into a numeric Weight_kg column by removing units and coercing invalid entries to NaN, then removing the small number of invalid rows.

In [32]:
```
df['Price'] = pd.to_numeric(df['Price'], errors='coerce').round(2)
```

The Price column was retained throughout preprocessing and treated as the target variable. It was converted to a numeric format and rounded to two decimal places, but otherwise not altered, as it already represents a clean continuous variable suitable for linear regression.

In [35]:
```
# helper function to split memory
def parse_memory(text):
    # remove empty spaces , make sure the value is i string and also chnage to u
    text = str(text).strip().upper()
    # split values with the plus sign
    parts = text.split('+')

    result = [] # store the gb and the storage type here ( list but values store
    # iterate through the split entries and remove empty spaces
    for item in parts:
        item = item.strip()
        if 'TB' in item:
            # convert TB to gb and extract gb
            size = float(item.split('TB')[0].strip()) * 1024
            # if storage type not available put UNKNOWN
            storage = item.split('TB')[1].strip() or 'UNKNOWN'
        elif 'GB' in item:
            # extract gb
            size = float(item.split('GB')[0].strip())
            #if storage type not available put UNKNOWN
            storage = item.split('GB')[1].strip() or 'UNKNOWN'
        else:
            # if size not present put 0
            size = 0
            # if storage not available but no
            storage = 'no'

        result.append((int(size), storage))
    # if there is one type of starage and gb for the memory , add and fill the e
    while len(result) < 2:
        # adding it to the result that already exits
        result.append((0, 'no'))
    #  return all of these : result[0][0] = storage size ,
    #[0][1] = storage type , [1][0] = extra storage size, [1][1] = extra storage
    return pd.Series([result[0][0], result[0][1], result[1][0], result[1][1]])

df = df.copy()
# for each row run parse_memory and write the output into the 4 columns respecti
df[['memory_gb', 'storage_type', 'extra_memory_gb', 'extra_storage_type']] = df[
# drop the column as it isnt needed anymore
df = df.drop(columns=['Memory'])

# check for rows that have all unknown values
bad_memory_rows = df[
    (df['memory_gb'] == 0) &
    (df['extra_memory_gb'] == 0) &
    (df['storage_type'] == 'no') &
    (df['extra_storage_type'] == 'no')
]
```

```
# drop the particular row
print("Rows with completely unparsed memory:", bad_memory_rows.shape[0])
print(bad_memory_rows)
df = df.drop(bad_memory_rows.index)
```

```
Rows with completely unparsed memory: 1
    Company  TypeName  Inches    ScreenResolution                          Cpu  \
770    Dell  Notebook    15.6  Full HD 1920x1080  Intel Core i7 7500U 2.7GHz

                    Gpu       OpSys     Price  Ram_GB  Weight_kg  memory_gb  \
770  AMD Radeon R7 M445  Windows 10  62938.07      16        2.3          0

    storage_type  extra_memory_gb extra_storage_type
770           no                0                 no
```

The Memory column was not in First Normal Form because some entries contained multiple storage devices in one field (e.g., '256GB SSD + 1TB HDD'). To convert this into a structured format suitable for regression, I parsed the column into four features: primary storage size (memory_gb), primary storage type (storage_type), secondary storage size (extra_memory_gb), and secondary storage type (extra_storage_type). TB values were converted to GB for consistent units. After extraction, the original Memory text column was removed because its information is now represented in separate 1NF columns.

In [37]:
```
# helper function to split cpu
def split_cpu(cpu):
    # if cpu values isnt a proper string or has ? set it to nan and no
    if not isinstance(cpu, str) or '?' in cpu:
        return pd.Series([np.nan, np.nan, 'no', np.nan])

     # split the text
    parts = cpu.strip().split()

    # if word is present get the word if not set it to nan
    brand = parts[0] if parts else np.nan

     # takes first word of the parts and keeps the word that has Ghz with it
    # first p is the output that we see the second p is the variable that holds
    speed = next((p for p in parts if 'GHz' in p), np.nan)
    # check speed
    if pd.notna(speed):
        # which index the speed is in
        speed_index = parts.index(speed)
        # model is an index before the speed
        model_word = parts[speed_index - 1] if speed_index > 0 else ''

        # check if the model has atleast 2 digits
        if bool(re.match(r'.*\d{2,}.*', model_word)):
            model = model_word
            # after brand before model . words inbetween is the line
            line_end = speed_index - 1
        else:
            # if it doesnt satisfy the 2 digits put no for mdel and use the rest
            model = 'no'
            line_end = speed_index


        line = ' '.join(parts[1:line_end])
```

```
    else:
        # if speed not found
        model_word = parts[-1] if parts else ''
        # check for 2 digits atleast
        if bool(re.match(r'.*\d{2,}.*', model_word)):
            model = model_word
            line = ' '.join(parts[1:-1])
        else:
            model = 'no'
            line = ' '.join(parts[1:])

        speed = np.nan

    return pd.Series([brand, line.strip(), model, speed])



df = df.copy()
# parse the data
df[['cpu_brand', 'cpu_line', 'cpu_model', 'cpu_speed_GHz']] = (
    df['Cpu'].apply(split_cpu)
)

# convert CPU speed to numeric type
df['cpu_speed_GHz'] = pd.to_numeric(
    df['cpu_speed_GHz'].astype(str).str.replace('GHz', '', regex=False),
    errors='coerce'
).round(1)

# remove rows with missing speed
df = df.dropna(subset=['cpu_speed_GHz'])

# drop CPU column
df = df.drop(columns=['Cpu'])


df[['cpu_brand', 'cpu_line', 'cpu_model', 'cpu_speed_GHz']].head(10)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1261 entries, 0 to 1302
Data columns (total 17 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Company             1261 non-null   object 
 1   TypeName            1261 non-null   object 
 2   Inches              1261 non-null   float64
 3   ScreenResolution    1261 non-null   object 
 4   Gpu                 1261 non-null   object 
 5   OpSys               1261 non-null   object 
 6   Price               1261 non-null   float64
 7   Ram_GB              1261 non-null   Int64  
 8   Weight_kg           1261 non-null   float64
 9   memory_gb           1261 non-null   int64  
 10  storage_type        1261 non-null   object 
 11  extra_memory_gb     1261 non-null   int64  
 12  extra_storage_type  1261 non-null   object 
 13  cpu_brand           1261 non-null   object 
 14  cpu_line            1261 non-null   object 
 15  cpu_model           1261 non-null   object 
 16  cpu_speed_GHz       1261 non-null   float64
dtypes: Int64(1), float64(4), int64(2), object(10)
memory usage: 178.6+ KB
```

The Cpu column was originally stored as an unstructured specification string (e.g., 'Intel Core i5 7200U 2.5GHz'), which is not directly usable in regression. I therefore parsed it into four separate attributes: CPU brand (cpu_brand), product line (cpu_line), model identifier (cpu_model), and clock speed (cpu_speed_GHz). Clock speed was converted into a numeric float by removing the 'GHz' suffix, because linear regression requires numerical predictors. After extraction, the original Cpu column was removed to avoid redundancy.

After parsing the CPU specifications, I identified rows where the CPU clock speed could not be extracted. Since clock speed is a key numerical predictor for price and required for linear regression, rows with missing cpu_speed_GHz were removed to avoid introducing noise or unreliable estimates into the model.

Some CPU model identifiers could not be extracted; however, clock speed was retained as the primary numeric indicator of CPU performance (model will have no if no value .

In [ ]:

In [39]:
```python
# helper function to split screen resolution
def split_screen(resolution):
    # handle missing or invalid entries
    if not isinstance(resolution, str) or '?' in resolution:
        return pd.Series([np.nan, np.nan, np.nan, False])

    resolution = resolution.strip().upper()

    # extract width and height digit space X space digit ( space optional)
    match = re.search(r'(\d+)\s*X\s*(\d+)', resolution)
    if match:
        width = int(match.group(1))
        height = int(match.group(2))
```

```
    else:
        # if not present put nan
        width, height = np.nan, np.nan

    # touchscreen present
    touchscreen = 'TOUCHSCREEN' in resolution

    # extract display type by removing unnessasary words
    display = re.sub(
        r'IPS PANEL|TOUCHSCREEN|\d+\s*X\s*\d+|/|\+',
        '',
        resolution
    ).strip()
    # if there isnt a display put nan
    display = display if display else np.nan
    # return 4 values to put in 4 columns
    return pd.Series([width, height, display, touchscreen])



df = df.copy()
# applying the parsing
df[['width', 'height', 'display', 'touchscreen']] = (
    df['ScreenResolution'].apply(split_screen)
)

# fill missing display values using company specific display mode, then global m
df['display'] = df.groupby('Company')['display'].transform(
    lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
)
# if cannot fill the display based on the company and the most used display , ge
global_display_mode = df['display'].mode().iloc[0]
df['display'] = df['display'].fillna(global_display_mode)  # IMPORTANT: assign b


# find rows where resolution could not be parsed (missing width height)
bad_screen_rows = df[df['width'].isna() | df['height'].isna()]

print("Rows with unparsed screen resolution:", bad_screen_rows.shape[0])
bad_screen_rows[['ScreenResolution', 'width', 'height', 'display', 'touchscreen'

# drop those rows
df = df.drop(bad_screen_rows.index)

# drop the column
df = df.drop(columns=['ScreenResolution'])

# quick check
df[['width', 'height', 'display', 'touchscreen']].head(10)
```

```
Rows with unparsed screen resolution: 0
```

Out[39]:

	width	height	display	touchscreen
0	2560	1600	RETINA DISPLAY	False
1	1440	900	RETINA DISPLAY	False
2	1920	1080	FULL HD	False
3	2880	1800	RETINA DISPLAY	False
4	2560	1600	RETINA DISPLAY	False
5	1366	768	FULL HD	False
6	2880	1800	RETINA DISPLAY	False
7	1440	900	RETINA DISPLAY	False
8	1920	1080	FULL HD	False
9	1920	1080	FULL HD	False

“ScreenResolution combined multiple attributes in one field (resolution, panel type, touchscreen). To satisfy 1NF and make the data usable for regression, I extracted numeric pixel dimensions (width, height) and a boolean touchscreen flag. The remaining descriptive label was stored as display. Missing display labels were imputed using the most common value within each brand (company mode) with a global mode fallback. Rows where width/height could not be parsed were removed because they are required numeric predictors

In [41]:

```
# helper function to split gpu
def split_gpu_info(text):
    # if text isnt a string or if it has nothing return placeholders with 3 outp
    if not isinstance(text, str) or text.strip() == '':
        return pd.Series(['no', 'no', 'no'])
    # remove spaces
    text = text.strip()

    # look for the brand INtel, AMD , Nvidia at the start of the text. if not th
    brand_search = re.search(r'^(Intel|AMD|Nvidia)', text, re.IGNORECASE)
    # get the matched brand if no put no
    brand = brand_search.group(0) if brand_search else 'no'

    # remove the brand part of the text to parse the rest of the text
    rest = text[len(brand):].strip() if brand != 'no' else text

     # GPU series patterns
    series_patterns = [
        'Iris Plus', 'Iris Pro', 'UHD', 'HD',
        'GeForce', 'Radeon', r'R\d'
    ]
    # combine the series pattern together , | means or
    series_regex = r'|'.join(series_patterns)
    # checks if rest has any of the options in the series pattern( ignore the up
    series_search = re.search(series_regex, rest, re.IGNORECASE)
    # extract the matched word if not put no ( group(0) is the whole match )
    series = series_search.group(0) if series_search else 'no'

    # split rest by series ( series is just the word)
```

```
    if series != 'no':
        model = re.split(re.escape(series), rest, flags=re.IGNORECASE)[-1]
        # remove the word graphics
        model = model.replace('Graphics', '').strip()
        model = model if model else 'no'
    else:
        # if no series is found just take whatever that is left as the model ( a
        model = rest.replace('Graphics', '').strip()
        model = model if model else 'no'

    return pd.Series([brand, series, model])



df = df.copy()
# apply the parsing
df[['gpu_brand', 'gpu_series', 'gpu_model']] = (
    df['Gpu'].apply(split_gpu_info)
)

# find rows with completely unparsed GPU information
bad_gpu_rows = df[
    (df['gpu_brand'] == 'no') &
    (df['gpu_series'] == 'no') &
    (df['gpu_model'] == 'no')
]

print("Rows with unparsed GPU info:", bad_gpu_rows.shape[0])
bad_gpu_rows[['Gpu', 'gpu_brand', 'gpu_series', 'gpu_model']].head(10)

# drop badly parsed GPU rows
df = df.drop(bad_gpu_rows.index)

# drop GPU column
df = df.drop(columns=['Gpu'])

#  check
df[['gpu_brand', 'gpu_series', 'gpu_model']].head(10)
```

```
Rows with unparsed GPU info: 0
```

Out[41]:

	gpu_brand	gpu_series	gpu_model
0	Intel	Iris Plus	640
1	Intel	HD	6000
2	Intel	HD	620
3	AMD	Radeon	Pro 455
4	Intel	Iris Plus	650
5	AMD	Radeon	R5
6	Intel	Iris Pro	no
7	Intel	HD	6000
8	Nvidia	GeForce	MX150
9	Intel	UHD	620

The Gpu column was stored as an unstructured string containing brand, series, and model information. To make this usable for analysis and encoding, I separated it into three attributes: GPU brand, GPU series, and GPU model. Rows where no meaningful GPU information could be extracted were removed to avoid introducing noise into the model. The original Gpu column was then dropped, as its information is now represented in structured form.

In [43]:
```
df.isna().sum()
```

Out[43]:
```
Company             0
TypeName            0
Inches              0
OpSys               0
Price               0
Ram_GB              0
Weight_kg           0
memory_gb           0
storage_type        0
extra_memory_gb     0
extra_storage_type  0
cpu_brand           0
cpu_line            0
cpu_model           0
cpu_speed_GHz       0
width               0
height              0
display             0
touchscreen         0
gpu_brand           0
gpu_series          0
gpu_model           0
dtype: int64
```

In [44]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1261 entries, 0 to 1302
Data columns (total 22 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Company             1261 non-null   object
 1   TypeName            1261 non-null   object
 2   Inches              1261 non-null   float64
 3   OpSys               1261 non-null   object
 4   Price               1261 non-null   float64
 5   Ram_GB              1261 non-null   Int64
 6   Weight_kg           1261 non-null   float64
 7   memory_gb           1261 non-null   int64
 8   storage_type        1261 non-null   object
 9   extra_memory_gb     1261 non-null   int64
 10  extra_storage_type  1261 non-null   object
 11  cpu_brand           1261 non-null   object
 12  cpu_line            1261 non-null   object
 13  cpu_model           1261 non-null   object
 14  cpu_speed_GHz       1261 non-null   float64
 15  width               1261 non-null   int64
 16  height              1261 non-null   int64
 17  display             1261 non-null   object
 18  touchscreen         1261 non-null   bool
 19  gpu_brand           1261 non-null   object
 20  gpu_series          1261 non-null   object
 21  gpu_model           1261 non-null   object
dtypes: Int64(1), bool(1), float64(4), int64(4), object(12)
memory usage: 219.2+ KB
```

After completing feature extraction, imputation, and data sanitisation, I rechecked the dataset for missing values. No remaining missing values were detected across any columns, confirming that the preprocessing steps successfully produced a complete dataset suitable for statistical analysis and linear regression modelling

Preprocessing Summary

The preprocessing stage converted raw laptop listing data into a structured and model ready dataset. Text based columns containing multiple attributes were decomposed into meaningful numeric and categorical features, missing values were handled through a combination of targeted imputation and justified row removal, and unrealistic values were filtered out. A final validation confirmed that no missing values remain in the dataset. This preprocessing ensures that the assumptions of linear regression are not violated by noisy or unstructured inputs and provides a reliable foundation for subsequent statistical analysis and modelling.

In [47]:
```
df.to_csv("cleands.csv", index=False)
```

The fully cleaned and processed dataset was saved locally as a CSV file (cleands.csv) to ensure reproducibility and to allow the modelling and evaluation stages to be carried out independently of the preprocessing pipeline.

Statistical analysis

Statistical analysis was performed on the cleaned numerical features to understand their central tendency (mean/median), spread (standard deviation and range), and distribution shape (skewness and kurtosis). This is important before fitting a linear regression model because regression coefficients can be sensitive to extreme values (outliers) and to predictors that have very different scales.

The analysis focuses on the following numerical variables: Price (INR), RAM (GB), screen size (Inches), weight (kg), primary and secondary storage capacity (memory_gb and extra_memory_gb), CPU speed (GHz), and screen resolution (width and height).

In [51]:
```
import pandas as pd
from scipy.stats import skew, kurtosis

# read clean csv
df = pd.read_csv("cleands.csv")
# statistical analysis on numerical features
numeric_features = [
    "Price", "Ram_GB", "Inches", "Weight_kg",
    "memory_gb", "extra_memory_gb", "cpu_speed_GHz",
    "width", "height"
]

stats_data = df[numeric_features]
# basic stats (mean, std, min, max and more)
summary_stats = stats_data.describe().T
# add skewness and kurtosis to the same DataFrame
summary_stats["skewness"] = stats_data.apply(skew)
summary_stats["kurtosis"] = stats_data.apply(kurtosis)
# show stats
summary_stats
```

Out[51]:

	count	mean	std	min	25%	50%	
Price	1261.0	59830.865575	37259.238810	9270.72	31914.72	52054.56	792
Ram_GB	1261.0	8.451229	5.561221	1.00	4.00	8.00	
Inches	1261.0	15.023553	1.424355	10.10	14.00	15.60	
Weight_kg	1261.0	2.077272	0.807686	0.00	1.50	2.04	
memory_gb	1261.0	449.135607	367.383271	8.00	256.00	256.00	5
extra_memory_gb	1261.0	173.151467	413.904160	0.00	0.00	0.00	
cpu_speed_GHz	1261.0	2.299128	0.507350	0.90	2.00	2.50	
width	1261.0	1893.571768	495.278508	1366.00	1600.00	1920.00	19
height	1261.0	1070.036479	284.854586	768.00	900.00	1080.00	10

The table summarises the statistics, skewness, and kurtosis for each numerical feature after preprocessing

Interpretation of results

PRICE

Price shows a positive skew of 1.54, showing that most laptops are within the low to mid price range, while a small number of expensive models create the long right tail. This is being reflected by the mean price (59830.87) being higher than the median (52,054.56). The positive kurtosis 4.50 highlights heavier tails than a normal distribution, showing the presence of extreme price outliers.

RAM

RAM is very positively skewed by 3.64 with very high kurtosis of 25.83. Most laptops have 4 to 8 GB of RAM, but a small number of high specification laptops (with 64GB) introduce strong outliers. Such values can disproportionately influence regression coefficients if its not scaled properly.

SCREEN SIZE INCHES

Screen size range (10.1 to 18.4 inches) and shows slight negative skew of 0.42, indicating a concentration around common laptop sizes, particularly 14 to 15.6 inches. This suggests that earlier preprocessing steps successfully removed unrealistic values, improving data quality for modelling.

WEIGHT

Weight is very positively skewed at 3.04 with heavy tails (kurtosis = 21.19). While most laptops weigh between 1.5 to 2.3kg, a small number of extreme values remain. The minimum value of 0.00 likely showing missing or incorrectly recorded weights, which may act as leverage points in regression and therefore require careful monitoring.

MEMORY_GB & EXTRA_MEMORY_GB

memory_gb is positively skewed at 1.56, with a strong range at 256 to 512GB. extra_memory_gb is even more skewed at 2.29, as alot of the laptops do not include additional storage, while a smaller part includes very large extra storage (up to 2048GB). This creates a distribution with many zeros that can affect regression stability.

CPU

CPU speed shows a moderate negative skew at 0.84, indicating that mid to high cpu speeds are more common than very low speeds. The low kurtosis suggests few extreme outliers compared to other features, making CPU speed a stable numeric predictor.

WIDTH & HEIGHT

Screen resolution variables (width and height) are positively skewed, with most laptops ranging around standard resolutions like 1920 x 1080. Higher resolution displays (eg. 3840 × 2160) create a right tail, which probably displays premium devices and supports their inclusion as meaningful predictors of price.

Overall, the dataset is suitable for linear regression because the target variable (Price) is continuous and the predictors represent meaningful, measurable hardware characteristics. However, the presence of strong skewness and high kurtosis in variables

such as RAM, storage, and weight indicates that feature scaling and outlier awareness are important to ensure stable coefficient estimation and reliable model performance.

These statistical findings informed the next stage of analysis, where visualisation is used to explore relationships between price and key predictors and to assess whether linear trends are visible prior to model fitting.

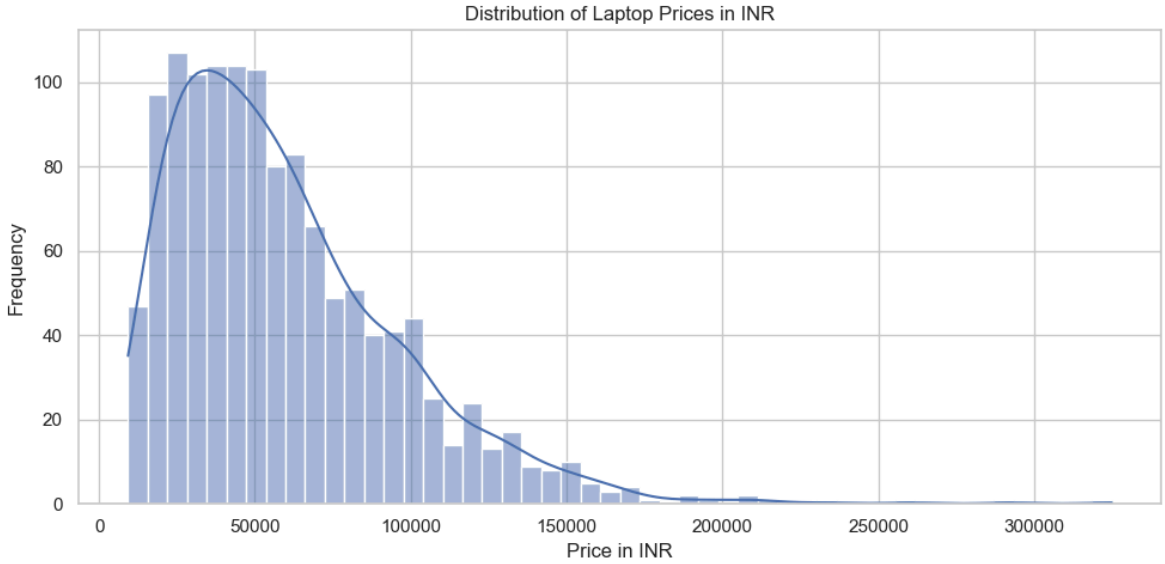
Visualisation

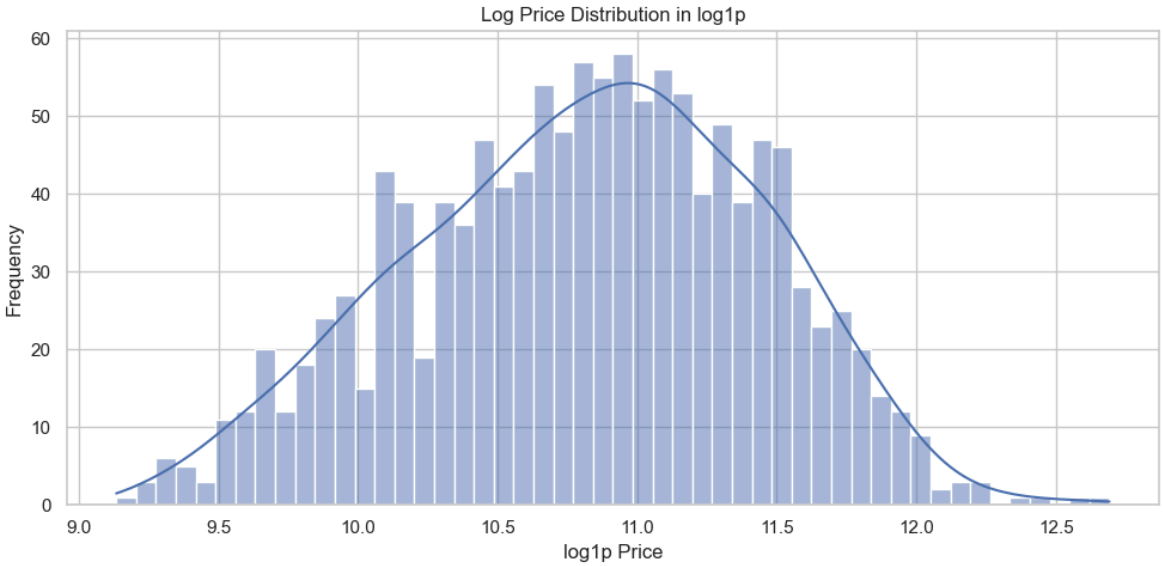
In [58]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [59]:
```
sns.set(style="whitegrid")
```

In [60]:
```
# distribution of the laptop price
# set figure size
plt.figure(figsize=(10,5))
# set bin to 50 and kde to true for a smoothed density curve
sns.histplot(df["Price"], bins=50, kde=True)
plt.title("Distribution of Laptop Prices in INR")
plt.xlabel("Price in INR")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# log price distribution (useful because price is heavily skewed)
# use log1p transformation to take care of skewness and zero values safely
# set plot size
plt.figure(figsize=(10,5))
# set bin to 50 and kde to true for a smoothed density curve
sns.histplot(np.log1p(df["Price"]), bins=50, kde=True)
# set title x and y label
plt.title("Log Price Distribution in log1p")
plt.xlabel("log1p Price")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```





The histogram of laptop prices shows a strong right skewed distribution, with the majority of laptops priced between approximately 30000 and 70000 Indian Rupees. This dense region reflects the mainstream consumer market, where most laptops are designed for everyday use such as study, office work, and general browsing. In contrast, there is a long and sparse right tail extending beyond 150000 and reaching up to around 325000. These extreme values correspond to a relatively small number of expensive devices, such as gaming laptops, professional workstations, and premium branded models.

An interesting observation is that while these premium laptops represent only a small part of the dataset, they have a disproportionately large influence on the overall price distribution. This explains why the mean price is notably higher than the median, as already identified in the statistical analysis. In practical terms, this means that most consumers are buying laptops far below the maximum price range, but the presence of a few very expensive models heavily stretches the scale of the data.

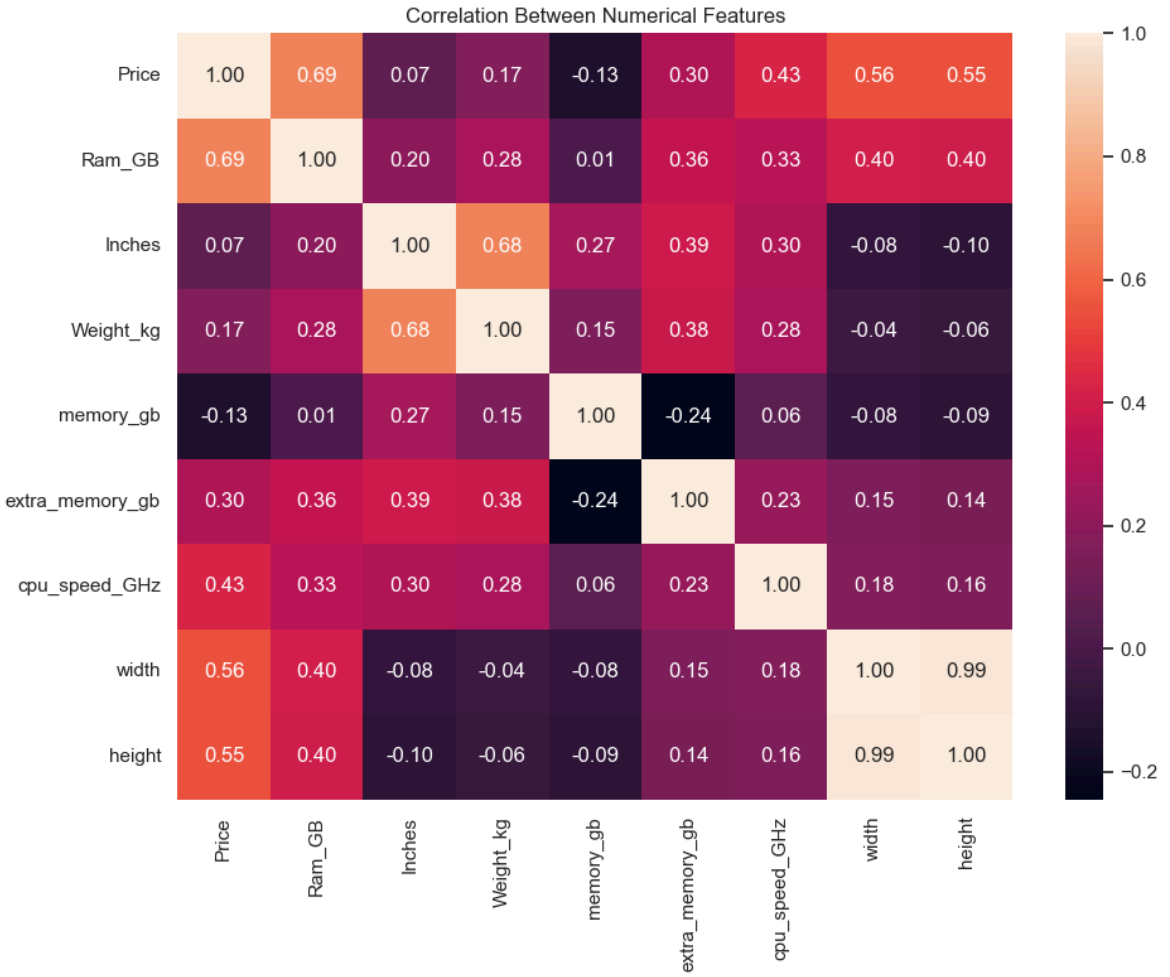
Since laptop prices in the dataset are strongly right skewed, with a small number of high end devices inflating the upper tail, I also visualised the target variable using a log1p (log(1 + x)) transformation. This transformation compresses extreme values while preserving the relative ordering of prices, making the overall distribution easier to interpret. The log1p form is numerically stable and avoids issues with zero values.

After applying the log1p transformation, the price distribution becomes much closer to symmetric, with a dominant peak and a more balanced spread on both sides. This confirms that the irregularity in the original distribution is not due to noise or data quality issues, but because of a large middle range segment combined with a small premium segment. The transformed plot makes this structure clearer by preventing the expensive laptops from visually dominating the distribution.

Although the regression model is trained using the original price values, this log transformed visualisation provides important insight into the data. It shows that price skewness is driven mainly by a small subset of premium laptops, highlighting the

importance of monitoring outliers and applying appropriate scaling techniques during linear regression modelling to avoid unwanted influence from extreme values.

In [62]:
```
# list of numerical features to put in the correlation matrix
numeric_features = ["Price","Ram_GB","Inches","Weight_kg","memory_gb","extra_mem
# compute the correlation matrix
corr = df[numeric_features].corr()
# set figure size
plt.figure(figsize=(10,8))
# plot the matrix as a heatmap
sns.heatmap(corr, annot=True, fmt=".2f")
# set title
plt.title("Correlation Between Numerical Features")
plt.tight_layout()
plt.show()
```



The correlation heatmap provides an overview of the linear relationships between the numerical features and laptop price.

The strongest correlation with Price is observed for RAM at 0.69, indicating that memory capacity is a main driver of laptop pricing in this dataset. This aligns with consumer expectations, as usually higher RAM configurations are typically associated with high performance oriented devices. However, the correlation is not close to 1, suggesting that RAM alone does not fully explain price variation and must be interpreted alongside other specifications.

Width at 0.56 and Height at 0.55, also shows a moderate positive correlation with price. This suggests that higher priced laptops tend to have higher resolution displays. An important and interesting observation is that width and height are almost perfectly correlated with each other at 0.99. This is expected because both features describe the same underlying characteristic which is screen resolution but it introduces a clear multicollinearity risk. Including both variables in a linear regression model could lead to unstable coefficient estimates, so one of them should be removed or replaced with a single derived feature (like total pixel count).

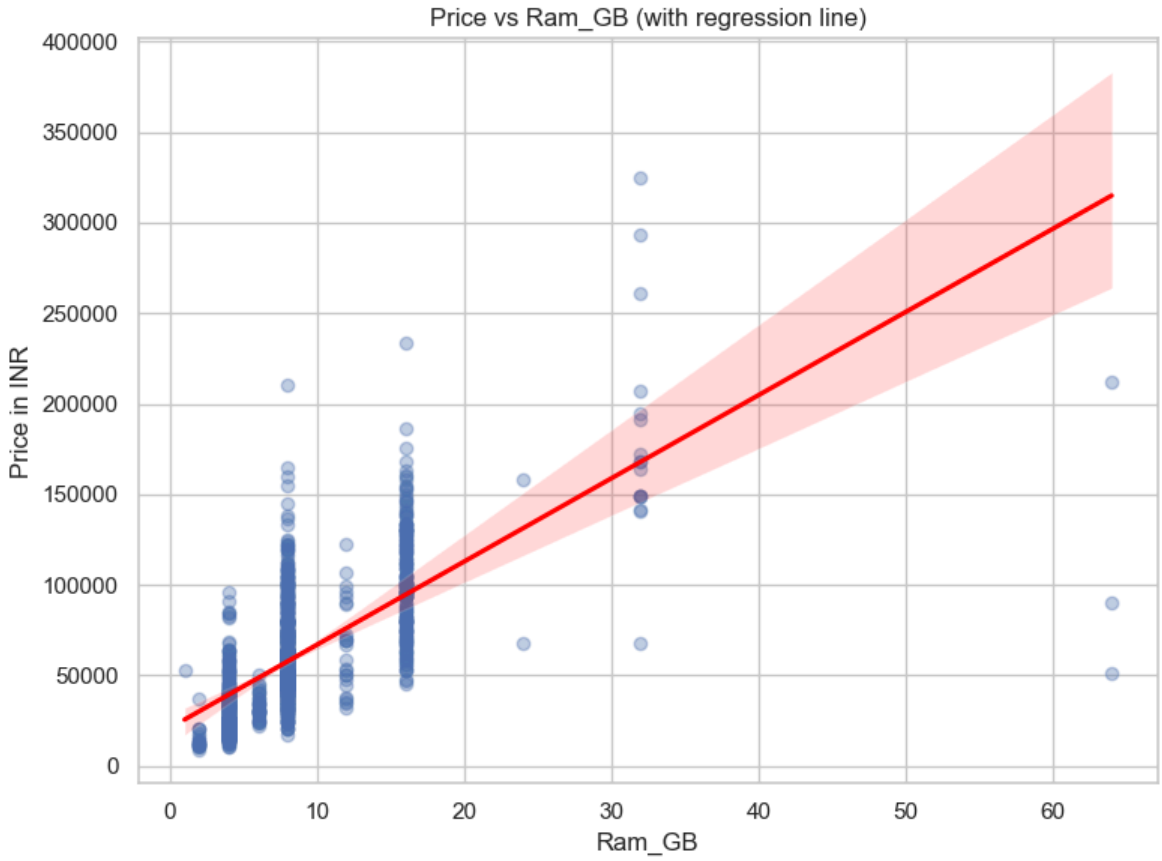
CPU speed shows a slightly negative correlation with price at 0.43), confirming that processing performance contributes to pricing but is not as dominant as RAM or display resolution. This supports the idea that CPU speed is a supporting predictor rather than a sole determinant of price.

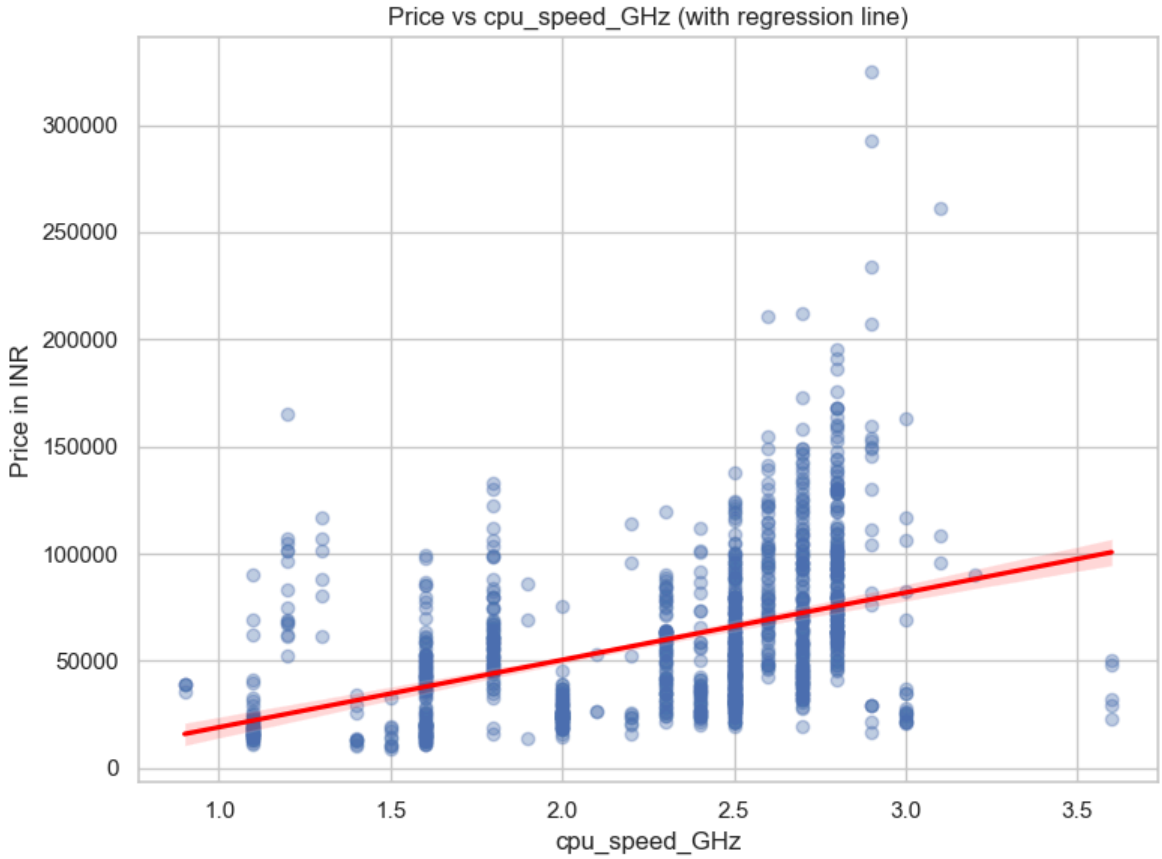
An interesting and less intuitive result is the weak or slightly negative correlation between base storage (memory_gb) and price (r = −0.13). This suggests that higher base storage capacity does not necessarily correspond to higher prices. In contrast, extra_memory_gb shows a positive correlation with price (r = 0.30), implying that expandable or secondary storage is more common in expensive or specialised devices. This distinction highlights why separating storage into base and extra components was important during preprocessing.

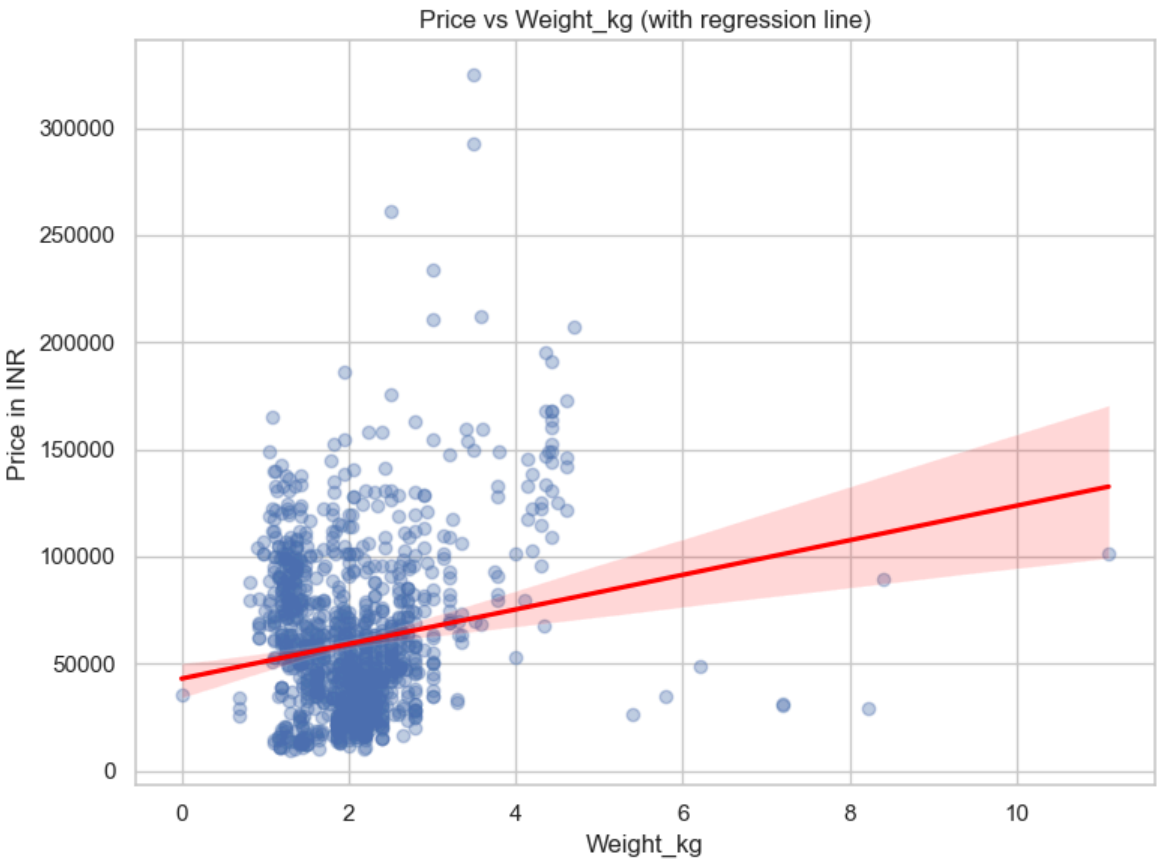
Physical characteristics like inches at 0.07 and weight at 0.17 show very weak and negative correlations with price. This indicates that larger or heavier laptops are not always more expensive, showing that performance related specifications dominate pricing rather than physical size alone.

Overall, the heatmap confirms that several predictors have meaningful linear relationships with price, supporting the use of linear regression. At the same time, it highlights key modelling considerations, particularly multicollinearity between width and height variables and the relatively weak correlation of some physical attributes. These insights directly inform feature selection and help justify modelling choices in later sections.

In [64]:
```
# select the important numerical features to analyse against price
core_plots = ["Ram_GB", "cpu_speed_GHz", "Weight_kg"]
# loop through each selected feature
for col in core_plots:
    # set the figure size
    plt.figure(figsize=(8,6))
    # draw a scatter plot with a fitted linear regression line
    sns.regplot(x=col, y="Price", data=df,scatter_kws={"alpha":0.35},line_kws={"
    # set the title x and y label
    plt.title(f"Price vs {col} (with regression line)")
    plt.xlabel(col)
    plt.ylabel("Price in INR")
    plt.tight_layout()
    plt.show()
```


Price vs Ram_GB (with regression line)
Price in INR
Ram_GB
400000
350000
300000
250000
200000
150000
100000
50000
0
0
10
20
30
40
50
60


Price vs cpu_speed_GHz (with regression line)
Price in INR
cpu_speed_GHz
300000
250000
200000
150000
100000
50000
0
1.0
1.5
2.0
2.5
3.0
3.5



The scatter plot of Price vs RAM shows a positive relationship, which is consistent with the high correlation observed earlier at 0.69. As RAM increases, the regression line rises sharply, indicating that RAM is one of the strongest numerical predictors of laptop price in the dataset.

A particularly interesting feature of this plot is the vertical clustering at common RAM values, especially at 4 GB to 8 GB, and around 16 GB. These clusters reflect standard configurations rather than continuous variation, meaning that RAM increases in discrete steps rather than smoothly. Within each cluster, there is still some price variation, showing that while RAM strongly influences price, it does not determine price on its own.

Another important observation is that the spread of prices increases at higher RAM , especially beyond 16 GB. Higher RAM laptops include both moderately priced devices and very expensive specialised models (eg. gaming or professional machines). These high RAM outliers pull the regression line upward and may have a strong influence coefficients. Overall, this plot visually confirms RAM as a key predictor while also highlighting the need to monitor outliers in linear regression.

The Price vs CPU speed plot shows a positive but a slight weak linear trend compared to RAM, with the correlation value at 0.43. The regression line slopes upward, indicating that faster processors are generally associated with higher prices, but the relationship is not that strong.

A notable feature is the large vertical spread of prices at similar CPU speeds, especially in the range around 2.3 and 3.0 GHz. Laptops with almost the same speeds can differ in price by almost over 100000. This suggests that CPU speed alone is an incomplete measure of processing capability.

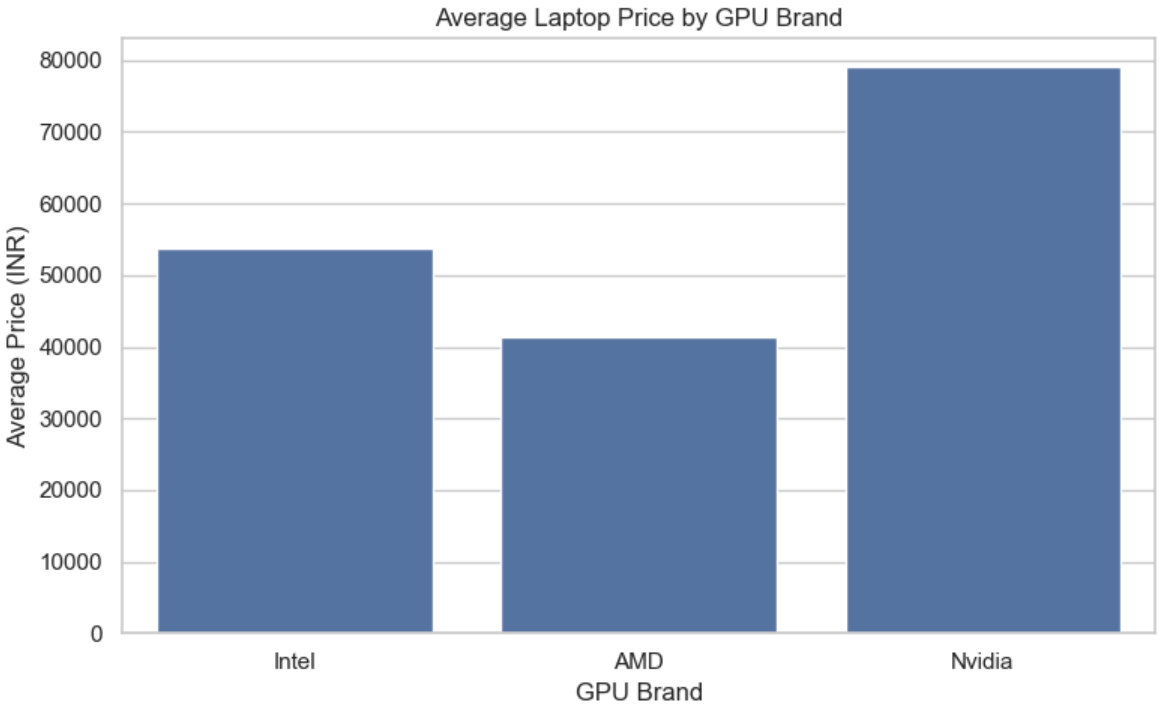
This plot highlights an important modelling insight: while CPU speed contributes useful information, its effect is likely to interact with other features such as RAM, GPU, and laptop type. In linear regression, this means CPU speed should be treated as a supportive predictor rather than a dominant one.

The Price vs Weight plot shows a weak positive trend, consistent with the low correlation observed earlier at 0.17. Most laptops are tightly clustered around 1 kg and 3 kg, covering a wide range of prices. This indicates that weight alone has limited explanatory power for pricing.

An observation is that very heavy laptops above 4 kg tend to be more expensive, but these points are rare. These heavier devices are likely gaming laptops or workstations that require extra hardware. However, because such models are not that frequent, they do not strongly influence the overall regression line.

This plot suggests that weight is more of an indirect indicator of laptop category rather than a direct driver of price. As a result, weight may add limited value as a standalone predictor in linear regression and could be more informative when combined with other features or categorical variables such as laptop type.

In [66]:
```
# filter out rows where the gpu brand is marked as "no"
gpu_filtered = df[df["gpu_brand"].str.lower() != "no"].copy()
# set plot size
plt.figure(figsize=(8,5))
# draw a bar plot showing the average laptop price for each gpu brand
sns.barplot(x="gpu_brand", y="Price", data=gpu_filtered, errorbar=None)
# set title x and y lable
plt.title("Average Laptop Price by GPU Brand")
plt.xlabel("GPU Brand")
plt.ylabel("Average Price (INR)")
# make sure everything fit within the figure
plt.tight_layout()
plt.show()
```



The bar chart shows a clear and substantial difference in average laptop price across GPU brands. Laptops with Nvidia GPUs have the highest average price, at around 79000, compared to 53000 for Intel and 41000 for AMD . This gap is large enough to highlight a difference in how these laptops are placed in the market.

An important insight from this plot is that GPU brand acts as a proxy for laptop category. Nvidia GPUs are typically dedicated graphics cards and are commonly found in gaming laptops, workstations, and other performance based devices. These laptops often include additional high end components such as larger RAM capacities, stronger CPU which together contribute to higher prices. In contrast, Intel and AMD GPUs are enough for everyday tasks and therefore appear more frequently in lower to mid priced laptops.

The lower average price of AMD laptops suggests that AMD GPUs are more widespread in budget configurations within this dataset, while Intel occupies a broader middle range segment. This distinction helps explain why GPU brand would contribute other than numerical specifications alone.

For a model, this visualisation highlights the importance of including GPU brand as a categorical feature in the regression model. While GPU brand itself is not a linear numeric variable, it does captures price differences that cannot be fully explained by continuous features such as RAM or CPU speed alone.

In [ ]:

The most important visualisation in this analysis is the Price vs RAM regression plot. This plot shows the clearest and strongest relationship between an individual feature and laptop price, which is also supported by the highest correlation value observed in the heatmap at 0.69. Unlike other predictors, RAM displays a consistent upward trend across the range of values, making it the most informative variable for explaining price variation.

Another insight from this plot is the groupings at standard RAM configurations like 4 GB, 8 GB, 16 GB, reflecting real world product design choices.The overall linear trend remains visible, which directly supports the use of linear regression as a baseline model. The spread of prices at higher RAM levels also highlights the influence of other factors, such as GPU and laptop category.

This visualisation is particularly important because it simultaneously justifies model choice, supports feature selection, and reveals key limitations of linear regression in a real world pricing context.

Build your ML (Machine Learning) model

After completing preprocessing, my dataset still contains a mix of numeric features (eg. Ram_GB, cpu_speed_GHz, width) and categorical text features (eg. Company, TypeName, OpSys, gpu_brand). Since linear regression requires numeric inputs, I converted the categorical columns into machine readable format and prepared the features to avoid common modelling issues such as sparse one hot columns and scale imbalance.

I removed cpu_model and gpu_model (if present). These fields tend to have very high cardinality (many unique values), which would increase the number of dummy variables during one hot encoding. That makes the matrix sparse and increases the risk of the model learning unwantted patterns tied to rare models instead of learning general pricing drivers (eg. RAM, GPU brand series, screen resolution).

In [71]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

In [72]:
```
# drop high cardinality columns (they create too many categories)
df = df.drop(columns=["cpu_model", "gpu_model"], errors="ignore")

# group rare categories into Other (reduces dummy columns)
min_count = 10

cpu_counts = df["cpu_line"].value_counts()
df.loc[df["cpu_line"].isin(cpu_counts[cpu_counts < min_count].index), "cpu_line"

company_counts = df["Company"].value_counts()
df.loc[df["Company"].isin(company_counts[company_counts < min_count].index), "Co
```

Even after cleaning, some categories appear only a few times like rare Company brands or cpu_line values. If I keep them, one hot encoding would create many dummy columns that represent only a handful of rows. That can lead to unstable coefficients and overfitting because the model can remember rare categories.

To reduce this from happening, I replaced categories with frequency < 10 with "Other":

> Company

> cpu_line

This keeps the model more robust and interpretable.

In [74]:
```
# one hot encode categorical columns
categorical_columns = [
    "Company", "TypeName", "OpSys",
    "cpu_brand", "gpu_brand", "display",
    "storage_type", "extra_storage_type",
    "cpu_line", "gpu_series", "touchscreen"
]
# we put drop_first = true to reduce redunancy and also have a reference column
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

# remove placeholder no columns if exist
df_encoded = df_encoded.drop(columns=[
    "gpu_brand_no", "gpu_series_no", "extra_storage_type_no", "storage_type_no"
], errors="ignore")
```

I used one hot encoding (pd.get_dummies) to convert categorical features into binary columns. I used drop_first=True so one category becomes the reference group, which avoids perfect multicollinearity (the dummy variable trap)

In [76]:
```
# define the target which is price which is y , and features which are the remai
y = df_encoded["Price"]
X = df_encoded.drop(columns=["Price"])

# train test split first to prevent scaling leakage
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# scale the features using the StandardScaler
# when the data is scaled it means that the mean is 0 and standard deviation is
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

The numeric features are on very different scales:

> width is in the thousands like 1920,
>
> price is in tens of thousands,
>
> cpu_speed_GHz is around 1 to 4,
>
> Ram_GB is around 4 to 16 commonly.

Without scaling, linear regression can still run, but the model becomes harder to compare and unstable when combining many numeric and dummy features. So I standardised the features with StandardScaler (mean 0, std 1). This makes training numerically stable and helps coefficient comparisons.

Training baseline Linear Regression and evaluating performance

I split the dataset into:

80% traininG 20% test with a fixed random seed of 42 for reproducibility.

Then I trained a baseline Linear Regression model and evaluated using:

RMSE: typical prediction error in currency units

R^2: proportion of variance explained (how much of the price variation the features capture)

In [78]:
```
# train the linear regression model
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
# predict and evaluate the model
y_pred = lr_model.predict(X_test_scaled)
# calculate the rmse and r2 to see how well the model has performed
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
# print the calculations
print("RMSE:", round(rmse, 2))
print("R²:", round(r2, 3))
```

```
RMSE: 18436.66
R²: 0.705
```

In [79]:
```
# check the coefficients of the top and bottom 10 features that affect the price
coef_df = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": lr_model.coef_
}).sort_values(by="Coefficient", ascending=False)

print("\nTop 10 positive coefficients (increase predicted price):")
display(coef_df.head(10))

print("\nTop 10 negative coefficients (decrease predicted price):")
display(coef_df.tail(10))
```

```
Top 10 positive coefficients (increase predicted price):
```

	Feature	Coefficient
28	OpSys_macOS	1.011524e+16
23	OpSys_Mac OS X	7.850917e+15
1	Ram_GB	1.268849e+04
6	width	1.026551e+04
5	cpu_speed_GHz	5.408596e+03
20	TypeName_Workstation	4.323500e+03
14	Company_Other	4.132688e+03
2	Weight_kg	3.575234e+03
3	memory_gb	3.560964e+03
11	Company_HP	3.537500e+03

```
Top 10 negative coefficients (decrease predicted price):
```

	Feature	Coefficient
46	cpu_line_Core i5	-2.725500e+03
45	cpu_line_Core i3	-3.463250e+03
21	OpSys_Chrome OS	-3.651500e+03
36	storage_type_HDD	-4.183000e+03
18	TypeName_Notebook	-4.804500e+03
22	OpSys_Linux	-4.969875e+03
35	display_RETINA DISPLAY	-5.135750e+03
24	OpSys_No OS	-7.392325e+03
25	OpSys_Windows 10	-7.739500e+03
8	Company_Apple	-1.275636e+16

In [80]:
```
# check the multicoliniarity for width and height and extract them iloc[0, 1]
if "width" in df.columns and "height" in df.columns:
    print("\nCorrelation (width vs height):",
          round(df[["width", "height"]].corr().iloc[0, 1], 3))
```

```
Correlation (width vs height): 0.994
```

RECAP: Feature and label selection and Model construction

The target variable for the regression model is Price (INR), as laptop price is a continuous numerical outcome suitable for linear regression. All remaining variables are treated as predictive features, including both numerical specifications (eg. RAM, CPU speed, screen resolution) and categorical attributes (eg. brand, laptop type, operating system).

Prior to modelling, several preprocessing decisions were made to ensure model stability and interpretability. Columns with extremely high cardinality, such as cpu_model and gpu_model, were removed because they introduce a large number of sparse dummy variables without adding proportional explanatory power. In addition, rare categories in Company and cpu_line that appeared fewer than ten times were grouped into an 'Other' category. This reduced dimensionality and overfitting caused by infrequent categories.

Categorical variables were converted into numerical form using one hot encoding, with the first category dropped in each case to avoid the dummy variable trap. Numerical features were standardised using StandardScaler so that all predictors had zero mean and unit variance, which is important for linear regression when features are measured on very different scales (eg. RAM in GB versus screen resolution in pixels).

Model evaluation

Model performance was evaluated using Root Mean Squared Error (RMSE) and R^2 on the untouched test set. The model achieved:

RMSE = 18436.66

$R^2 = 0.705$

An R^2 value of 0.705 indicates that approximately 70.5% of the variance in laptop prices is explained by the selected features. Given the wide price range in the dataset from entry level laptops to high end devices this represents strong performance for a linear model applied to real world consumer data. The RMSE suggests that, on average, predictions deviate from the true price by around 18436.66, which is reasonable relative to the overall scale of prices.

Interpretation of coefficients

Inspection of the learned coefficients reveals that several features act as strong positive predictors of price. RAM capacity and screen resolution (width) have the largest positive coefficients among numerical features, confirming earlier findings from the statistical analysis and visualisations that performance related specifications dominate pricing. CPU speed also contributes positively, indicating that faster processors are associated with higher priced laptops.

Categorical features provide additional insight. Workstation laptops are predicted to be more expensive than the baseline laptop type, which aligns with their professional use cases. Similarly, certain brands and GPU categories show positive price effects after controlling for other specifications.

Some coefficients, notably Company_Apple and OpSys_macOS, have extremely large magnitudes with opposite signs. This behaviour is caused by multicollinearity, as Apple laptops almost exclusively run macOS. The model struggles to separate the effect of brand from operating system, resulting in unstable coefficient estimates. Importantly, this issue affects coefficient understandability rather than the predictive performance, as reflected by the stable R^2 and RMSE values.

Multicollinearity considerations

Multicollinearity is also observed between screen width and height, which have a correlation of 0.994. Both variables encode the same underlying screen resolution and including both in the model increases coefficient magnitudes without really significantly improving predictions. This confirms the need for feature selection or dimensionality reduction in later modelling stages.

Summary

Overall, the linear regression model performs well and captures the dominant drivers of laptop pricing, particularly RAM, CPU speed, display resolution, and laptop type. While multicollinearity among some categorical and resolution features limits coefficient interpretability, the model provides a strong and interpretable baseline that is suitable for further validation and improvement using regularisation or alternative feature representations.

In [ ]:

Validation

In [83]:
```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import  Ridge, Lasso

# list of models to cross validate and scale the data first then train
models = {
    "Linear Regression": Pipeline([("scaler", StandardScaler()),("model", Linear
    "Ridge Regression": Pipeline([("scaler", StandardScaler()),("model", Ridge(a
    "Lasso Regression": Pipeline([("scaler", StandardScaler()),("model", Lasso(a
}

# perform  5 fold cross validation
for name, pipeline in models.items():
    scores = cross_val_score(pipeline,X,y,cv=5,scoring="r2")
    print(f"{name} CV R² scores: {np.round(scores, 3)}")
    print(f"{name} Mean CV R²: {scores.mean():.3f}")
    print("-" * 40)
```

```
Linear Regression CV R² scores: [ 6.28000000e-01 -7.80774643e+21  7.32000000e-01
6.15000000e-01
 -9.26302301e+22]
Linear Regression Mean CV R²: -2008759529834351375 1552.000
----------------------------------------
Ridge Regression CV R² scores: [0.63  0.746 0.733 0.618 0.741]
Ridge Regression Mean CV R²: 0.694
----------------------------------------
Lasso Regression CV R² scores: [0.629 0.747 0.732 0.618 0.74 ]
Lasso Regression Mean CV R²: 0.693
----------------------------------------
C:\Users\vinay\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_desce
nt.py:697: ConvergenceWarning: Objective did not converge. You might want to incr
ease the number of iterations, check the scale of the features or consider increa
sing regularisation. Duality gap: 2.512e+08, tolerance: 1.386e+08
  model = cd_fast.enet_coordinate_descent(
```

Cross validation reveals that ordinary least squares linear regression is unstable across different data splits, producing extremely negative R^2 values in some folds. This behaviour is caused by strong multicollinearity in the dataset, particularly between highly correlated numerical features (such as screen width and height) and categorical variables that encode overlapping information (eg. Company and operating system).

Rather than correcting or removing these effects at this stage, the instability of linear regression is intentionally retained to demonstrate its limitations as a baseline model. Ridge and Lasso regression address this issue by introducing regularisation, which stabilises coefficient estimates and improves generalisation.

The consistent cross validated performance of Ridge and Lasso, with mean R^2 values of approximately 0.69, confirms that regularised linear models are more appropriate for this dataset. This validates the use of regularisation as a principled response to multicollinearity rather than a data preprocessing flaw.

In [ ]:
```
```

Feature engineering

In [86]:
```
from sklearn.compose import ColumnTransformer # [4]
from sklearn.preprocessing import OneHotEncoder, StandardScaler, PolynomialFeatu

# load cleaned dataset
df = pd.read_csv("cleands.csv").copy()

# drop the high cardinality text columns if  exist
df = df.drop(columns=[c for c in ["cpu_model", "gpu_model"] if c in df.columns])

# split X and y
y = df["Price"]
X = df.drop(columns=["Price"])

# group the columns data based on the numeric and the categorical according to t
num_cols = ["Ram_GB","Inches","Weight_kg","memory_gb","extra_memory_gb","cpu_spe
cat_cols = ["Company","TypeName","OpSys","cpu_brand","cpu_line","gpu_brand","gpu
            "display","storage_type","extra_storage_type","touchscreen"]

# keep only columns that exist
num_cols = [c for c in num_cols if c in X.columns]
cat_cols = [c for c in cat_cols if c in X.columns]

# train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# pipeline , standarddize the numerical columns and one hot encode the categoric
baseline_preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), cat_cols)
])
# build the ridge pipeline
ridge_base = Pipeline([
    ("prep", baseline_preprocess),
    ("ridge", Ridge(alpha=1.0))
])
# train the ridge model
ridge_base.fit(X_train, y_train)
pred_base = ridge_base.predict(X_test)
# generate the predictions on the unseen test set
rmse_base = np.sqrt(mean_squared_error(y_test, pred_base))
r2_base = r2_score(y_test, pred_base)
# print the results
print("Baseline Ridge RMSE:", round(rmse_base, 2))
print("Baseline Ridge R²:", round(r2_base, 3))

# build pipeline for polynomial
poly_preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("poly", PolynomialFeatures(degree=2, include_bias=False)),
        ("scale", StandardScaler())
    ]), num_cols),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), cat_cols)
])
# polynomial features in ridge
ridge_poly = Pipeline([
    ("prep", poly_preprocess),
    ("ridge", Ridge(alpha=1.0))
```

```
])

# train the model
ridge_poly.fit(X_train, y_train)
pred_poly = ridge_poly.predict(X_test)
# evaluate the results
rmse_poly = np.sqrt(mean_squared_error(y_test, pred_poly))
r2_poly = r2_score(y_test, pred_poly)
# print the results
print("\nPolynomial Ridge (numeric only) RMSE:", round(rmse_poly, 2))
print("Polynomial Ridge (numeric only) R²:", round(r2_poly, 3))
```

```
Baseline Ridge RMSE: 17660.08
Baseline Ridge R²: 0.729

Polynomial Ridge (numeric only) RMSE: 17373.82
Polynomial Ridge (numeric only) R²: 0.738
```

A controlled feature engineering experiment was conducted by introducing polynomial features on numeric variables only, while keeping categorical features one hot encoded. Ridge regression was used to ensure stability and isolate the effect of feature engineering.

Compared to the Ridge model with an RMSE of 17660 and R^2 of 0.729, the polynomial Ridge model achieved a lower RMSE of 17374 and a higher R^2 of 0.738. This indicates a moderate but consistent improvement in predictive performance.

The results suggest that while laptop pricing is largely driven by linear relationships, certain numeric features such as screen resolution show mild non linear effects that can be captured through polynomial expansion. Applying polynomial features selectively avoids unnecessary feature increase and prevents the instability observed when polynomial transformations were applied randomly.

Overall, this confirms that careful, feature engineering can provide measurable gains without sacrificing model stability or interpretability.

Evaluation of your model

This project explored how well linear regression algorithm could predict laptop prices based on technical specifications. After cleaning the data, removing rare and high cardinality values, and applying one hot encoding and scaling, I trained and tested several models to evaluate their performance.

The final model performance was evaluated using Root Mean Squared Error (RMSE) and the coefficient of determination (R^2). Among the models tested, the Polynomial Ridge regression with polynomial features applied only to numeric variables produced the strongest results, achieving an RMSE of approximately 17374 and an R^2 of 0.738. This indicates that around 73.8% of the variance in laptop prices is explained by the selected features. Baseline Ridge regression without polynomial features performed slightly worse at around R^2 0.729, while standard linear regression was shown to be unstable under cross validation due to multicollinearity and high dimensional one hot encoded features.

RMSE was chosen as the primary evaluation metric because laptop price is a continuous numerical target where prediction errors are naturally interpreted in currency units. RMSE is particularly appropriate in this context because it penalises large errors more heavily, which is important when expensive laptops exist in the dataset and large under or over predictions would not be wanted. This makes RMSE more informative than absolute error measures for pricing problems.

From a reflective perspective, the project demonstrates that careful preprocessing and regularisation are critical when working with real world retail data. Although basic linear regression achieved reasonable performance on a single train test split, cross validation revealed instability caused by multicollinearity (for example, between screen width and height) and a large number of dummy variables. Ridge and Lasso regression addressed this issue by shrinking coefficients and improving generalisation. Feature engineering using polynomial terms initially worsened performance when applied to all features, but restricting polynomial expansion to numeric variables only led to a measurable improvement, showing that controlled feature engineering can be beneficial when applied selectively.

This project provides a structured and interpretable approach to laptop price prediction, highlighting which specifications most strongly influence cost. The method can be used on other areas like smartphones, vehicles, or real estate, where mixed numerical and categorical data are common and regularised regression models are well suited. Overall, the project demonstrates a robust and well justified machine learning pipeline that balances model performance, interpretability, and generalisability.

REFERENCES

[1] SADIQ, E. (2023). Uncleaned Laptop Price dataset. [online] Kaggle.com. Available at: https://www.kaggle.com/datasets/ehtishamsadiq/uncleaned-laptop-price-dataset/data [Accessed 7 Jan. 2026].

[2] James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). An Introduction to Statistical Learning with Applications in R Second Edition. [online] Available at: https://www.casact.org/sites/default/files/2022-12/James-G.-et-al.-2nd-edition-Springer-2021.pdf.

[3] chmx0929 (2016). UIUCclasses/512DataMiningPrinciples/Data Mining - Concepts and Techniques [3rd ed.] - Jiawei Han, Micheline Kamber, Jian Pei.pdf at master · chmx0929/UIUCclasses. [online] GitHub. Available at: https://github.com/chmx0929/UIUCclasses/blob/master/512DataMiningPrinciples/Data%20M%20Concepts%20and%20Techniques%20%5B3rd%20ed.%5D%20-%20Jiawei%20Han%2C%20Micheline%20Kamber%2C%20Jian%20Pei.pdf.

[4] scikit-learn.org. (n.d.). sklearn.compose.ColumnTransformer — scikit-learn 0.23.2 documentation. [online] Available at: https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html.

In [ ]:

In [ ]: